

Промежуточный отчёт по лабораторной работе №6 по предмету “Технологии управления данными”

Текст задачи:

Лабораторная работа №6. Создание скрипта восстановления утраченных филиалом данных на основании детальных данных Хранилища.

Цель работы: Создание скрипта, обеспечивающего обратный перенос очищенных данных из Хранилища в выбранный Филиал.

Скрипт можно оформить как набор SQL команд, или как хранимую процедуру. Скрипт должен переносить данные за выбранный промежуток времени из Хранилища в Филиал. Скрипт должен работать исходя из того, что данные в Филиале полностью отсутствуют.

Решение:

Промежуток времени рассматривается по `AddedDate`.

`last_added_date` - последняя актуальная дата добавления информации в таблицы продуктов и категорий

`last_relevant_{entity}` - ID последней (по дате) актуальной сущности (покупатель, чек) в филиале (через `Store{Entity}ID`)

Условие добавления в филиал:

Факт покупки товара добавляется в филиал, если он сделан актуальным покупателем по актуальному чеку.

Добавляются только актуальные покупатели, чеки, а также каталог товаров.

Это исключает дублирование информации, при этом обеспечивая актуальность данных в филиалах

Код скрипта (на Python):

```
import datetime
import psycopg2
```

```
db_config_west = {
    "dbname": "west_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5446"
}
```

```
db_config_east = {
    "dbname": "east_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5447"
}
```

```
db_config_warehouse = {
    "dbname": "warehouse_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5448"
}
```

```
conn_warehouse = None
conn = None
```

```
try:
    start_date = datetime.datetime.strptime(input(f"Start date with format
YYYY-MM-DD: "), '%Y-%m-%d')
    end_date = datetime.datetime.strptime(input(f"  End date with format
YYYY-MM-DD: "), '%Y-%m-%d')
    configs = [db_config_east, db_config_west]
    stores = {"east": "0", "west": "1"}
    storeID = stores[input(f"Choose store (east/west): ")]
    config = configs[int(storeID)]
    conn = psycopg2.connect(**config)
    conn_warehouse = psycopg2.connect(**db_config_warehouse)
    with conn_warehouse.cursor() as cur_warehouse:
        product_map = dict()
        receiptIDs = set()
        with conn.cursor() as cur:
            cur_warehouse.execute("SELECT AddedDate "
                                   "FROM Product "
                                   "WHERE StoreID = %s AND AddedDate >= %s
AND AddedDate <= %s "
                                   "ORDER BY AddedDate DESC "
                                   "LIMIT 1",
                                   (str(storeID), start_date, end_date))
            last_added_date = cur_warehouse.fetchone()[0]
            cur_warehouse.execute("SELECT ProductID, StoreProductID,
ProductName, StoreListedPrice "
                                   "FROM Product "
                                   "WHERE AddedDate = %s AND StoreID = %s",
                                   (last_added_date, storeID))
            products = cur_warehouse.fetchall()
            for product in products:
                productID = str(product[0])
                pID = str(product[1])
                name = product[2]
                price = str(product[3])
                cur.execute("INSERT INTO Product (ProductID, ProductName,
ProductPrice) "
                            "VALUES (%s, %s, %s)",
                            (pID, name, price))
                product_map[productID] = pID
            cur_warehouse.execute("SELECT CategoryID, StoreCategoryID,
CategoryName "
                                   "FROM Category "
                                   "WHERE AddedDate = %s AND StoreID =
%s",
```

```

        (last_added_date, storeID))
    categories = cur_warehouse.fetchall()
pcID = 0
for category in categories:
    categoryID = str(category[0])
    cID = str(category[1])
    name = category[2]
    cur.execute("INSERT INTO Category (CategoryID, CategoryName)
"
        "VALUES (%s, %s)",
        (cID, name))
    cur_warehouse.execute("SELECT ProductID "
        "FROM ProductCategory "
        "WHERE AddedDate = %s AND CategoryID =
%s",
        (last_added_date, categoryID))
    product_categories = cur_warehouse.fetchall()
    for product_category in product_categories:
        productID = str(product_category[0])
        pID = product_map[productID]
        cur.execute("INSERT INTO ProductCategory
(ProductCategoryID, CategoryID, ProductID) "
            "VALUES (%s, %s, %s)",
            (pcID, cID, pID))
        pcID += 1
    cur_warehouse.execute("SELECT StoreCustomerID "
        "FROM Customer "
        "WHERE StoreID = %s AND AddedDate >= %s
AND AddedDate <= %s",
        (storeID, start_date, end_date))
    customers = cur_warehouse.fetchall()
    riID = 0
    receiptID = -1
    for customer in customers:
        cID = str(customer[0])
        cur_warehouse.execute("SELECT CustomerID, CustomerFirstName,
CustomerLastName "
            "FROM Customer "
            "WHERE StoreCustomerID = %s AND
StoreID = %s AND AddedDate >= %s AND AddedDate <= %s "
            "ORDER BY AddedDate DESC "
            "LIMIT 1",
            (cID, str(storeID), start_date,
end_date))
        last_relevant_customer = cur_warehouse.fetchone()
        customerID = str(last_relevant_customer[0])
        firstName = last_relevant_customer[1]
        lastName = last_relevant_customer[2]

```

```

        cur.execute("INSERT INTO Customer (CustomerID,
CustomerFirstName, CustomerLastName) "
                    "VALUES (%s, %s, %s)",
                    (cID, firstName, lastName))
    cur_warehouse.execute("SELECT DISTINCT ReceiptID "
                           "FROM SoldItem "
                           "WHERE StoreID = %s AND CustomerID =
%s AND AddedDate >= %s AND AddedDate <= %s "
                           "ORDER BY ReceiptID DESC",
                           (storeID, customerID, start_date,
end_date))

    sold_item_receiptIDs = cur_warehouse.fetchall()
    for sold_item_receiptID in sold_item_receiptIDs:
        receiptID = str(sold_item_receiptID[0])
        cur_warehouse.execute("SELECT StoreReceiptID "
                               "FROM Receipt "
                               "WHERE ReceiptID = %s ",
                               (receiptID,))

        rID = cur_warehouse.fetchone()[0]
        cur_warehouse.execute("SELECT ReceiptID, TotalPrice,
CompletedDate "
                               "FROM Receipt "
                               "WHERE StoreReceiptID = %s AND
StoreID = %s AND AddedDate >= %s AND AddedDate <= %s "
                               "ORDER BY AddedDate DESC "
                               "LIMIT 1",
                               (rID, str(storeID), start_date,
end_date))

        last_relevant_receipt = cur_warehouse.fetchone()
        receiptID = str(last_relevant_receipt[0])
        total_price = last_relevant_receipt[1]
        completed_date = last_relevant_receipt[2]
        if receiptID not in receiptIDs:
            cur.execute("INSERT INTO Receipt (ReceiptID,
CustomerID, TotalPrice, CompletedDate) "
                        "VALUES (%s, %s, %s, %s)",
                        (rID, cID, total_price, completed_date))
            receiptIDs.add(receiptID)
            cur_warehouse.execute("SELECT ProductID,
ActualPrice, Amount "
                                   "FROM SoldItem "
                                   "WHERE ReceiptID = %s AND
StoreID = %s AND AddedDate >= %s AND AddedDate <= %s",
                                   (receiptID, str(storeID),
start_date, end_date))

            sold_items = cur_warehouse.fetchall()
            for sold_item in sold_items:
                productID = str(sold_item[0])

```

```

        actual_price = sold_item[1]
        amount = sold_item[2]
        try:
            cur.execute("INSERT INTO ReceiptItem
(ReceiptItemID, ReceiptID, ProductID, ActualPrice, Amount) "
                        "VALUES (%s, %s, %s, %s, %s)",
                        (riID, rID, product_map[productID],
actual_price, amount))

            riID += 1
        except KeyError:
            print("Skipped purchase of no longer
relevant product", productID)
            conn.commit()
except (Exception, psycopg2.DatabaseError) as error:
    if conn:
        conn.rollback()
        print("Error with PostgreSQL:", error)
except ValueError as error:
    print("Invalid date.", error)
except KeyError as error:
    print("Invalid store", error)
finally:
    if conn_warehouse:
        conn_warehouse.close()
    if conn:
        conn.close()
print("All done!")

```

*Для удобства переноса данных в сущность Товар хранилища данных добавлено (избыточное) поле “цена”.